perplexity

it feels this is more for reference and not in the perfect learning order. look i wanna become a master and the best developer out there. no more nice to know. we are gonna know everything or atleast be aware of them. arrange the topics in the right learning order so that going through them will be optimal. no removing any topic, I want all 77 - just rearrange them to make sense. give the same labels so i know which ones to focus and which ones to just glance over

No searching needed — all 77 topics are already captured. This is a pure reordering operation based on dependency logic: you learn what unlocks the next thing first, always building upward. Every topic stays, every label stays.

The Rule of This Order

Each topic assumes you fully own everything above it. You do not skip down and come back. You move linearly, top to bottom, like clearing a building floor by floor.

Complete Python Backend Mastery — Optimal Learning Order (All 77 Topics)

Order	Topic	Subtopics	Label
1	Dev environment & workflow	Python versions, `pyenv`, virtual environments, `venv`, package installers, REPL/IPython, running scripts, debugging basics, editor setup, shell usage, `pip`, `pipx`, project bootstrapping	Must know
2	Linux & networking	Processes, signals, ports, sockets, DNS, HTTP/TCP basics, file permissions, systemd awareness, inspecting logs, basic troubleshooting on a server	Should know
3	Git & collaboration	Branching, rebasing, merge discipline, pull requests, code reviews, semantic commits, release notes, resolving conflicts, reviewing like an engineer	Must know
4	Syntax & primitives	Variables, naming, literals, integers, floats, booleans, strings, `None`, operators, expressions, truthiness, unpacking, multiple assignment	Must know

Order	Topic	Subtopics	Label
5	Control flow	`if/elif/else`, `for`, `while`, loop control, loop `else`, comprehensions, ternary expressions, walrus operator, branching patterns	Must know
6	Strings	Indexing, slicing, immutability, formatting, f-strings, encoding/decoding, common methods, multiline strings, raw strings, parsing patterns	Must know
7	Core collections	Lists, tuples, sets, dictionaries, mutability rules, copying, sorting, hashing, membership, nested structures, collection methods	Must know
8	Functions	Parameters, defaults, positional vs keyword args, `*args`, `**kwargs`, return values, pure vs impure functions, recursion, lambdas, higher-order functions	Must know
9	Scope & namespaces	LEGB, local vs global, closures, `nonlocal`, `global`, variable shadowing, namespace lookup, side effects from shared state	Must know
10	File I/O & paths	Reading/writing files, binary vs text mode, `pathlib`, streams, buffering, temporary files, safe file handling patterns	Must know
11	Serialization & data formats	JSON, CSV, YAML basics, `pickle` risks, custom encoders, schema awareness, file upload/download concepts, validating external data	Must know
12	Exceptions & error handling	Built-in exceptions, custom exceptions, `try/except/else/finally`, exception chaining, re-raising, handling errors at boundaries, designing meaningful error types	Must know
13	Standard library essentials	`os`, `sys`, `pathlib`, `datetime`, `uuid`, `enum`, `functools`, `itertools`, `collections`, `contextlib`, `hashlib`, `secrets`, `subprocess`, `tempfile`	Must know
14	Dates, time, and time zones	`datetime`, naive vs aware datetimes, UTC storage, timezone conversions, scheduling hazards, serialization of timestamps	Must know
15	Regex & text processing	Pattern syntax, groups, named groups, validation, extraction, substitution, greedy vs non-greedy matching, when regex is the right tool	Should know
16	OOP fundamentals	Classes, instances, attributes, methods, constructors, inheritance, composition, encapsulation, polymorphism, class vs instance variables, `classmethod`, `staticmethod`	Must know
17	Python data model	Dunder methods, `__repr__`, `__str__`, `__len__`, `__iter__`, `__getitem__`, `__contains__`, comparison methods, callable objects, operator overloading	Should know
18	Dataclasses & properties	`@dataclass`, frozen dataclasses, default factories, `property`, getters/setters, value objects, lightweight domain modeling	Should know
19	Abstract classes & protocols	`abc`, abstract base classes, interfaces by convention, `typing.Protocol`, structural subtyping, designing stable contracts between layers	Should know
20	Type hints & static typing	Basic annotations, generics, `Optional`, unions, `Literal`, `TypedDict`, `TypeVar`, `Callable`, forward refs, runtime vs static typing, `mypy` mindset	Must know

Order	Topic	Subtopics	Label
21	Iterators & generators	Iterable vs iterator, `iter`, `next`, generator functions, generator expressions, lazy evaluation, `yield from`, memory-efficient pipelines	Must know
22	Functional patterns	`map`, `filter`, `reduce`, `partial`, first-class functions, callables, function composition, when functional style helps and when it hurts readability	Should know
23	Decorators	Function decorators, class decorators, decorator factories, stacking decorators, preserving metadata with `functools.wraps`, real backend use cases such as auth, logging, retries, caching	Should know
24	Context managers	`with`, `__enter__`, `__exit__`, `contextlib`, resource cleanup, transaction boundaries, file/database/network safety patterns, async context managers	Should know
25	Advanced collections	`deque`, `Counter`, `defaultdict`, `namedtuple`, `OrderedDict`, `ChainMap`, `heapq`, choosing the right structure for the workload	Should know
26	Modules & packages	`import`, absolute vs relative imports, package layout, `__init__.py`, circular imports, reusable structure, public vs private APIs, import side effects	Must know
27	Packaging & distribution	`pyproject.toml`, dependencies, lockfiles, build backends, packaging an app vs a library, entry points, installable internal tools, versioning basics	Should know
28	Linters, formatters & quality gates	`black`, `ruff`, `mypy`, import sorting, `pre-commit`, CI checks, failing builds on quality issues, enforcing standards automatically	Must know
29	Complexity & CS foundations	Big-O, common algorithmic tradeoffs, arrays/lists, stacks, queues, heaps, trees, graphs, hashing, searching, sorting, choosing structures intentionally	Must know
30	Clean code & refactoring	Naming, modularity, cohesion, coupling, small functions, removing duplication, simplifying control flow, code smells, safe refactors, readability first	Must know
31	Design principles	SOLID, separation of concerns, dependency inversion, composition over inheritance, stable boundaries, avoiding overengineering	Should know
32	Python execution model	How Python runs code, interpreter vs script mode, bytecode, `.pyc`, module loading, `__name__ == "__main__"`, object references, identity vs equality, mutability, reference counting, garbage collection	Should know
33	Debugging & introspection	`pdb`, stack traces, breakpoints, traceback reading, runtime inspection, profiling, memory inspection, reproducing bugs, writing minimal failing cases	Must know
34	Configuration & secrets	Environment variables, settings modules, secret rotation basics, per-environment config, twelve-factor concepts, avoiding config drift	Must know

Order	Topic	Subtopics	Label
35	Async Python	Event loop, `async`/`await`, coroutines, tasks, `gather`, cancellation, timeouts, async context managers, async iterators, structured concurrency concepts	Must know
36	Concurrency & parallelism	Threads, processes, executors, queues, locks, semaphores, CPU-bound vs I/O-bound work, GIL implications, choosing the right concurrency model	Should know
37	HTTP fundamentals	Requests/responses, methods, headers, status codes, cookies, sessions, content types, idempotency, caching headers, TLS basics, proxies, reverse proxies	Must know
38	ASGI/WSGI & web runtime	ASGI vs WSGI, app server vs web server, Uvicorn, Gunicorn, worker models, connection handling, timeouts, process model, reverse proxy flow	Must know
39	Security fundamentals	Input validation, injection prevention, secrets management, CSRF/CORS basics, secure headers, dependency risk, SSRF awareness, deserialization risks, least privilege	Must know
40	Cryptography basics	Hashing, salting, password storage, HMAC, signatures, token signing, when to use crypto libraries and when not to invent anything yourself	Should know
41	SQL & PostgreSQL	SQL syntax, joins, aggregations, transactions, constraints, indexes, views, normalization, denormalization tradeoffs, query plans, locking basics	Must know
42	Database design	Entity modeling, one-to-one/one-to-many/many-to-many, primary and foreign keys, unique constraints, check constraints, soft deletes, audit fields, schema evolution	Must know
43	SQLAlchemy	ORM vs Core, models, sessions, relationships, query building, eager/lazy loading, N+1 awareness, async support, raw SQL escape hatches	Must know
44	Alembic & migrations	Migration creation, upgrade/downgrade flow, schema drift, safe migrations, data migrations, zero-downtime considerations, migration review habits	Must know
45	Pydantic & validation	Schema modeling, nested models, validators, settings management, serialization/deserialization, strictness, field constraints, API contract discipline	Must know
46	FastAPI core	Routing, path/query params, request bodies, response models, dependencies, middleware, exception handlers, lifespan events, background tasks, file uploads	Must know
47	Authentication	Password hashing, sessions, token auth, JWT basics, OAuth2 flows, refresh tokens, API keys, secure credential handling, login/logout patterns	Must know
48	Authorization	Roles, permissions, claims, policy checks, object-level authorization, least privilege, separating authn from authz in code design	Must know
49	API design	Resource modeling, REST conventions, pagination, filtering, sorting, partial updates, versioning, error format, consistency rules, backward compatibility	Must know

Order	Topic	Subtopics	Label
50	Testing fundamentals	Unit tests, integration tests, end-to-end tests, Arrange-Act-Assert, fixtures, parametrization, mocks, fakes, stubs, test isolation, coverage meaning	Must know
51	Testing async & APIs	Async test clients, DB test setup, transactional test isolation, dependency overrides, contract testing, testing failure modes, API schema tests	Must know
52	Logging	Structured logs, log levels, correlation IDs, request logging, masking secrets, log sampling, using logs for diagnosis instead of noise	Must know
53	Documentation	README quality, API docs, architecture diagrams, ADRs, docstrings, operational runbooks, onboarding guides, documenting edge cases and tradeoffs	Must know
54	Docker	Dockerfiles, image layers, slim images, multi-stage builds, containers vs VMs, local dev containers, environment injection, persistent volumes basics	Must know
55	CI/CD	Automated tests, build pipelines, image publishing, deployment workflows, environment promotion, rollback basics, migration automation, release hygiene	Must know
56	Redis & caching	Cache-aside pattern, TTLs, invalidation, distributed locks basics, session storage, rate limiting helpers, ephemeral state, hot-path optimization	Should know
57	Background jobs & task queues	Celery-style workers, brokers, retries, dead-letter concepts, scheduled jobs, outbox pattern basics, non-blocking workload separation	Should know
58	Webhooks & integrations	Designing inbound/outbound webhooks, signature verification, retries, replay protection, idempotency keys, integration failure handling	Should know
59	WebSockets, SSE & real-time patterns	Persistent connections, pub/sub patterns, connection lifecycle, fan-out, state management, when WebSockets beat polling, backpressure awareness	Should know
60	Observability	Metrics, traces, health checks, SLO thinking, latency/error-rate visibility, dashboards, alerts, root-cause workflow, observability by design	Should know
61	Performance & profiling	CPU vs I/O bottlenecks, benchmarking, profiling, query optimization, caching strategy, memory hotspots, pagination efficiency, load-test thinking	Should know
62	Resilience patterns	Retries, exponential backoff, circuit breakers, timeouts, bulkheads, graceful degradation, idempotency, failure budgets, compensating actions	Should know
63	Architecture patterns	Layered architecture, clean architecture, ports and adapters, repository pattern, service layer, unit of work, DTOs, dependency injection patterns	Should know
64	Domain modeling	Ubiquitous language, aggregates, invariants, workflows, transactional boundaries, translating business rules into code cleanly, avoiding anemic models	Should know

Order	Topic	Subtopics	Label
65	System design	Scalability, stateless services, horizontal vs vertical scaling, partitioning, consistency vs availability tradeoffs, queues, caching, bottleneck analysis	Should know
66	Distributed systems	Network failure, partial failure, clock issues, retries causing duplicates, eventual consistency, service boundaries, coordination costs, backpressure	Should know
67	Messaging & event-driven systems	RabbitMQ/Kafka/NATS concepts, producers/consumers, ordering, delivery guarantees, idempotent consumers, eventual consistency, event choreography basics	Should know
68	Kubernetes for backend engineers	Deployments, Services, Ingress, ConfigMaps, Secrets, probes, resource requests/limits, rolling updates, app-level debugging, not cluster-admin depth	Should know
69	Cloud fundamentals	Compute, storage, networking, managed databases, IAM basics, load balancers, queues, serverless tradeoffs, cost awareness, environment setup	Should know
70	gRPC & alternative API styles	gRPC basics, protobuf concepts, schema-first contracts, when RPC beats REST, internal service communication patterns	Nice to know
71	NoSQL, search & object storage	MongoDB/DynamoDB use cases, Elasticsearch/OpenSearch basics, blob/object storage, when not to force everything into PostgreSQL	Nice to know
72	Multi-tenancy & product patterns	Tenant isolation, row-level strategies, quota enforcement, feature flags, audit trails, billing hooks, per-tenant config, operational implications	Nice to know
73	Memory optimization	`__slots__`, generator pipelines, object lifetime, avoiding accidental copies, streaming responses, large-file handling, memory-aware design	Nice to know
74	Advanced testing strategy	Contract tests, property-based testing, mutation testing awareness, load and chaos test concepts, regression suites for production bugs	Nice to know
75	Advanced Python internals	Descriptors, metaclasses, MRO depth, import hooks, monkey patching risks, descriptors behind properties/method binding, advanced introspection	Nice to know
76	Metaprogramming	Dynamic class creation, decorators at scale, registries, plugin systems, reflection, code generation patterns, when not to get clever	Nice to know
77	Leadership & engineering judgment	Tradeoff writing, estimating risk, picking boring technology when appropriate, incident ownership, mentoring, making systems maintainable for a team	Should know

The Progression Logic (Why This Order)

- **Topics 1–3** — before writing a single line, your machine works, your shell works, and your version control works. Everything else depends on this.
- **Topics 4–15** — pure Python language. No frameworks, no databases. Just the language itself from the ground up.
- **Topics 16–28** — advanced language features. OOP, types, async building blocks, and project structure. You need the basics solid before OOP makes deep sense.
- **Topics 29–34** — CS foundations, clean code, principles, execution model, debugging, and config. You learn *how to think* before you touch a framework.
- **Topics 35–36** — async and concurrency go here because FastAPI is async-first. You cannot use FastAPI properly without owning these.
- **Topics 37–49** — the entire backend production stack: HTTP, runtime, security, database, FastAPI, auth, API design. This is the core of the job.
- **Topics 50–55** — testing, logging, docs, Docker, and CI/CD. These are locked in immediately after you can build, because shipping untested unobservable software is worthless.
- **Topics 56–69** — advanced backend systems: caching, jobs, messaging, real-time, resilience, architecture, system design, distributed systems, Kubernetes, cloud. These compound everything below.
- **Topics 70–77** — the ceiling. NoSQL, gRPC, metaprogramming, advanced testing, memory, internals, multi-tenancy, and engineering leadership. This is what separates senior from elite.